# Servidor Echo TCP (RFC 862) em Java

Explicação do código `App.java`, seguida do código limpo (sem comentários) pronto para copiar.

## 1. O que o programa faz

Este programa é um **servidor TCP** que implementa o protocolo **Echo** descrito na RFC 862. A regra do protocolo é simples: tudo o que o cliente enviar deve ser devolvido exatamente igual, byte por byte. O servidor escuta na porta **12345** (a RFC sugere a porta 7, mas ela costuma exigir privilégios de administrador) e atende vários clientes ao mesmo tempo, usando uma thread para cada conexão.

## 2. Imports

- `ServerSocket` e `Socket`: o primeiro fica à espera de conexões; o segundo representa a conexão com um cliente específico.
- `InputStream` e `OutputStream`: fluxos de bytes para ler e escrever no socket.
- `ExecutorService` e `Executors`: gerenciam o pool de threads.
- `IOException`: exceção lançada em falhas de entrada e saída de rede.

## 3. Atributos da classe

- `PORT = 12345`: porta TCP onde o servidor escuta.
- `threadPool = Executors.newCachedThreadPool()`: cria threads sob demanda e reaproveita as que ficaram ociosas. Assim, cada cliente é atendido em paralelo sem travar os outros.

## 4. Método main

O `main` abre o `ServerSocket` dentro de um *try-with-resources* (ele é fechado automaticamente ao sair do bloco) e entra em um loop infinito:

- `serverSocket.accept()` bloqueia até um cliente se conectar e devolve o `Socket` dessa conexão.
- `threadPool.submit(() -> handleClient(clientSocket))` entrega o atendimento a outra thread, e o loop volta imediatamente a aceitar novos clientes.

Se ocorrer um erro no servidor principal, a mensagem é exibida e o bloco `finally` chama `threadPool.shutdown()`, que impede novas tarefas e deixa as em andamento terminarem.

## 5. Método handleClient

Executa dentro de uma thread do pool, uma por cliente.

- **Buffer de 4 KB**: ler e escrever em blocos é mais eficiente do que byte a byte.
- **try-with-resources**: obtém `in` e `out` do socket e os fecha automaticamente no final.
- **Loop de eco**: `in.read(buffer)` devolve quantos bytes foram lidos, ou **-1** quando o cliente termina de enviar (EOF). Enquanto não for -1, `out.write(buffer, 0, bytesRead)` devolve exatamente os bytes recebidos, sem converter para texto, e `flush()` garante o envio imediato.
- **Encerramento**: ao receber -1 o loop termina, o servidor imprime uma mensagem e fecha a conexão.

- **catch**: erros de comunicação, como cliente caindo de repente, são registrados em `System.err` sem derrubar o servidor.
- **finally**: tenta fechar o socket de novo por garantia. Como os streams já foram fechados (o que fecha o socket), isso é redundante, mas inofensivo.

## 6. Fluxo resumido

| Etapa | O que acontece |
|---|---|
| 1 | Servidor abre a porta 12345 e espera. |
| 2 | Cliente conecta e accept() devolve um Socket. |
| 3 | O Socket é enviado a uma thread do pool. |
| 4 | A thread lê bytes e os reenvia ao cliente, repetidamente. |
| 5 | Cliente fecha o envio (EOF), a thread fecha a conexão e termina. |

## 7. Como testar

Compile e execute o servidor, e em outro terminal conecte com o netcat ou o telnet:

```
javac App.java
java App

nc localhost 12345
```

Tudo o que você digitar será devolvido na tela. Para encerrar, use Ctrl+D (EOF) no netcat.

## 8. Pontos de atenção

- O `newCachedThreadPool` não limita o número de threads. Muitos clientes simultâneos podem esgotar a memória; em produção, considere um pool de tamanho fixo.
- Não há timeout. Um cliente conectado que não envia nada mantém uma thread ocupada. Dá para usar `clientSocket.setSoTimeout(...)`.
- Se `accept()` lançar IOException, o servidor inteiro encerra.

# Código sem comentários (para copiar)

Salve como `App.java`.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class App {
    private static final int PORT = 12345;
    private static final ExecutorService threadPool = Executors.newCachedThreadPool();

    public static void main(String[] args) {
        System.out.println("Iniciando o servidor RFC 862 Echo na porta " + PORT + "...");

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                threadPool.submit(() -> handleClient(clientSocket));
            }
        } catch (IOException e) {
            System.err.println("Erro no servidor principal: " + e.getMessage());
        } finally {
            threadPool.shutdown();
        }
    }

    private static void handleClient(Socket clientSocket) {
        byte[] buffer = new byte[4096];
        int bytesRead;

        try (
            InputStream in = clientSocket.getInputStream();
            OutputStream out = clientSocket.getOutputStream()
        ) {
            while ((bytesRead = in.read(buffer)) != -1) {
                out.write(buffer, 0, bytesRead);
                out.flush();
            }
            System.out.println("Cliente encerrou o envio. Fechando conexão de forma limpa.");

        } catch (IOException e) {
            System.err.println("Erro durante a comunicação com o cliente: " + e.getMessage());
        } finally {
            try {
                clientSocket.close();
            } catch (IOException e) {
                System.err.println("Erro ao fechar o socket do cliente: " + e.getMessage());
            }
        }
    }
}
```